

Asignatura: Desarrollo de Aplicaciones Web (IS093A)

Unidad II: Desarrollo Web Fullstack

Tema: Diseño de APIs RESTful con Django REST Framework (DRF): Serialización, ViewSets, Routers, Paginación, Filtrado, Throttling, CORS/CSRF e HATEOAS Modalidad: Laboratorio

APELLIDOS Y NOMBRES:

Alarcón Mendoza Estiven Rodrigo

PARTE 1. Instalación

1. Crear y activar entorno virtual

- `python -m venv venv`
- `venv\Scripts\activate`

Instalar dependencias

- `pip install django`
- `pip install djangorestframework`
- `pip install django-filter`
- `pip install django-cors-headers`
- `pip install drf-spectacular`

Guardar dependencias

- `pip freeze > requirements.txt`

2. Configurar settings.py

Agregar aplicaciones

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
  
    'rest_framework',  
    'corsheaders',  
    'drf_spectacular',  
    'django_filters',  
  
    'productos',  
]
```

Agregar middleware

```
MIDDLEWARE = [  
    'django.middleware.security.SecurityMiddleware',  
    'django.contrib.sessions.middleware.SessionMiddleware',  
    'django.middleware.common.CommonMiddleware',  
    'django.middleware.csrf.CsrfViewMiddleware',  
    'django.contrib.auth.middleware.AuthenticationMiddleware',  
    'django.contrib.messages.middleware.MessageMiddleware',  
    'django.middleware.clickjacking.XFrameOptionsMiddleware',  
    'corsheaders.middleware.CorsMiddleware',  
]
```

Configurar CORS

```
CORS_ALLOWED_ORIGINS = [  
    "http://localhost:3000",  
    "http://127.0.0.1:3000",  
]
```

Configurar DRF

```
REST_FRAMEWORK = {  
  
    "DEFAULT_SCHEMA_CLASS":  
        "drf_spectacular.openapi.AutoSchema",  
  
    "DEFAULT_PAGINATION_CLASS":  
        "rest_framework.pagination.PageNumberPagination",  
  
    "PAGE_SIZE": 5,  
  
    "DEFAULT_FILTER_BACKENDS": [  
        "django_filters.rest_framework.DjangoFilterBackend",  
        "rest_framework.filters.SearchFilter",  
        "rest_framework.filters.OrderingFilter",  
    ],  
  
    "DEFAULT_THROTTLE_CLASSES": [  
        "rest_framework.throttling.AnonRateThrottle",  
        "rest_framework.throttling.UserRateThrottle",  
    ],  
  
    "DEFAULT_THROTTLE_RATES": {  
        "anon": "10/min",  
        "user": "100/min",  
    }  
}
```

Configurar Swagger

```
SPECTACULAR_SETTINGS = {  
  
    "TITLE": "API Productos",  
    "DESCRIPTION": "API REST con Django REST Framework",  
    "VERSION": "1.0.0",  
  
}
```

Verificar

python manage.py check

```
• (venv) PS D:\IX semestre\Desarrollo Web\Semana 13\S13 API> python manage.py check  
System check identified no issues (0 silenced).
```

PARTE 2. Crear modelo

models.py

```
from django.db import models  
class Producto(models.Model):  
    nombre = models.CharField(max_length=150)  
    precio = models.DecimalField(max_digits=10, decimal_places=2)  
    stock = models.IntegerField()  
    creado = models.DateTimeField(auto_now_add=True)  
  
    def __str__(self):  
        return self.nombre
```

Migraciones

- python manage.py makemigrations
- python manage.py migrate

PARTE 3. Serialización Avanzada

Crear: serializers.py

```
from rest_framework import serializers  
  
from .models import Producto  
  
class ProductoSerializer(serializers.HyperlinkedModelSerializer):  
    precio_con_igv = serializers.SerializerMethodField()
```

```

class Meta:
    model = Producto
    fields = (
        'url',
        'id',
        'nombre',
        'precio',
        'precio_con_igv',
        'stock',
        'creado',
    )
    read_only_fields = (
        'creado',
    )

    def get_precio_con_igv(self, obj):
        return round(float(obj.precio) * 1.18, 2)

    def validate_stock(self, value):
        if value < 0:
            raise serializers.ValidationError(
                'El stock no puede ser negativo.'
            )

        return value

```

Se utiliza

- HyperlinkedModelSerializer
- SerializerMethodField
- Validación personalizada
- ReadOnlyField

PARTE 4. ViewSets

views.py

```

from rest_framework import viewsets

from .models import Producto
from .serializers import ProductoSerializer

class ProductoViewSet(viewsets.ModelViewSet):
    queryset = Producto.objects.all()
    serializer_class = ProductoSerializer

```

```
filterset_fields = ['stock']
search_fields = ['nombre']
ordering_fields = ['precio', 'creado']
```

Métodos generados automáticamente

No necesitamos escribir:

- GET
- POST
- PUT
- PATCH
- DELETE

porque ModelViewSet crea:

- list()
- retrieve()
- create()
- update()
- partial_update()
- destroy()

PARTE 5. Router

urls.py de la aplicación

```
from rest_framework.routers import DefaultRouter

from .views import ProductoViewSet

router = DefaultRouter()
router.register(
    r'productos',
    ProductoViewSet,
    basename='producto',
)

urlpatterns = router.urls
```

urls.py principal

```
from django.contrib import admin
from django.urls import include, path
from django.views.generic import RedirectView
from drf_spectacular.views import SpectacularAPIView
```

```

from drf_spectacular.views import SpectacularSwaggerView

urlpatterns = [
    path('', RedirectView.as_view(url='/api/docs/', permanent=False)),
    path('admin/', admin.site.urls),
    path('api/', include('productos.urls')),

    path(
        "api/schema/",
        SpectacularAPIView.as_view(),
        name="schema"
    ),

    path(
        "api/docs/",
        SpectacularSwaggerView.as_view(
            url_name="schema"
        ),
        name="swagger"
    ),
]

```

Ahora existen automáticamente

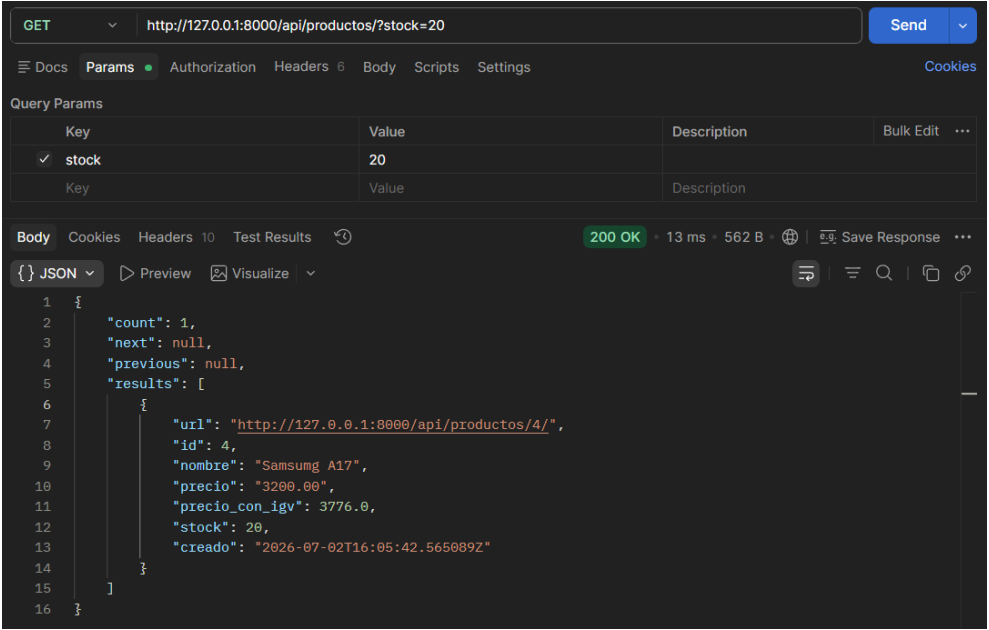
- GET /api/productos/
- POST /api/productos/
- GET /api/productos/1/
- PUT /api/productos/1/
- PATCH /api/productos/1/
- DELETE /api/productos/1/

PARTE 6. Filtrado

Ejemplo

GET

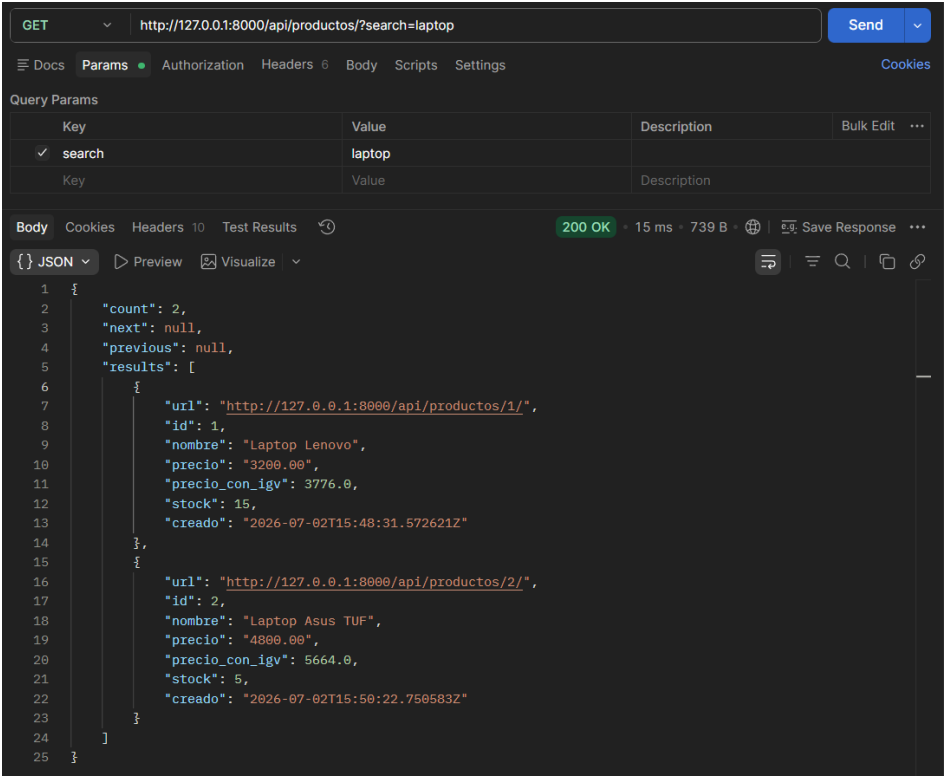
/api/productos/?stock=20



Búsqueda

GET

/api/productos/?search=laptop



Ordenamiento

GET

/api/productos/?ordering=precio

GET

http://127.0.0.1:8000/api/productos/?ordering=precio

Send

Docs

Params

Authorization

Headers 6

Body

Scripts

Settings

Cookies

Query Params

Key	Value	Description	Bulk Edit	...
ordering	precio			
Key	Value	Description		

Body

Cookies

Headers 10

Test Results

200 OK

15 ms

1.05 KB

Save Response

...

JSON

Preview

Visualize

...

```
1 {
2   "count": 4,
3   "next": null,
4   "previous": null,
5   "results": [
6     {
7       "url": "http://127.0.0.1:8000/api/productos/1/",
8       "id": 1,
9       "nombre": "Laptop Lenovo",
10      "precio": "3200.00",
11      "precio_con_igv": 3776.0,
12      "stock": 15,
13      "creado": "2026-07-02T15:48:31.572621Z"
14    },
15    {
16      "url": "http://127.0.0.1:8000/api/productos/3/",
17      "id": 3,
18      "nombre": "Samsung",
19      "precio": "3200.00",
20      "precio_con_igv": 3776.0,
21      "stock": 15,
22      "creado": "2026-07-02T16:04:50.846331Z"
23    },
24    {
25      "url": "http://127.0.0.1:8000/api/productos/4/",
26      "id": 4,
27      "nombre": "Samsung A17",
28      "precio": "3200.00",
29      "precio_con_igv": 3776.0,
```

PARTE 7. Paginación

Página 2

/api/productos/?page=2

GET

http://127.0.0.1:8000/api/productos/?page=2

Send

Docs

Params

Authorization

Headers 6

Body

Scripts

Settings

Cookies

Query Params

Key	Value	Description	Bulk Edit	...
page	2			
Key	Value	Description		

Body

Cookies

Headers 10

Test Results

404 Not Found

16 ms

371 B

Save Response

...

JSON

Preview

Debug with AI

...

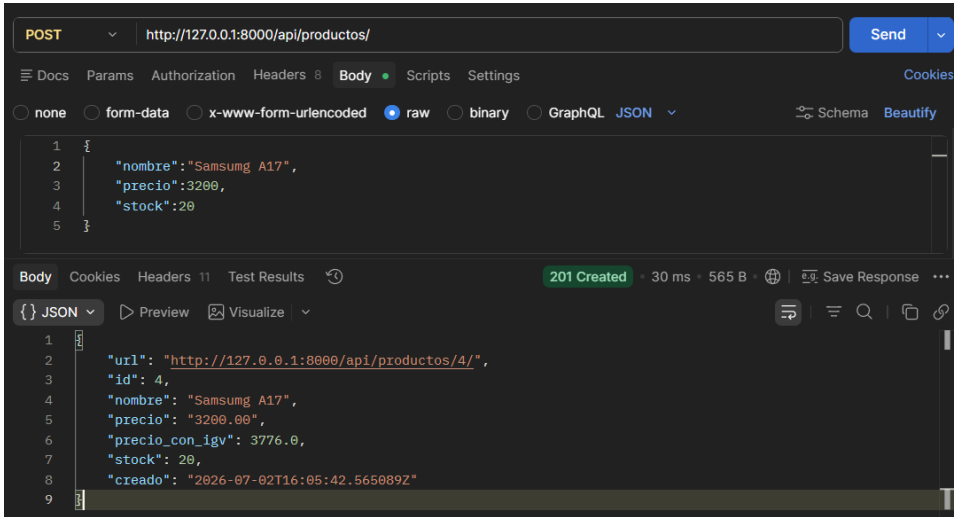
```
1 {
2   "detail": "Invalid page."
3 }
```


PARTE 8. Pruebas en Postman

Crear

POST

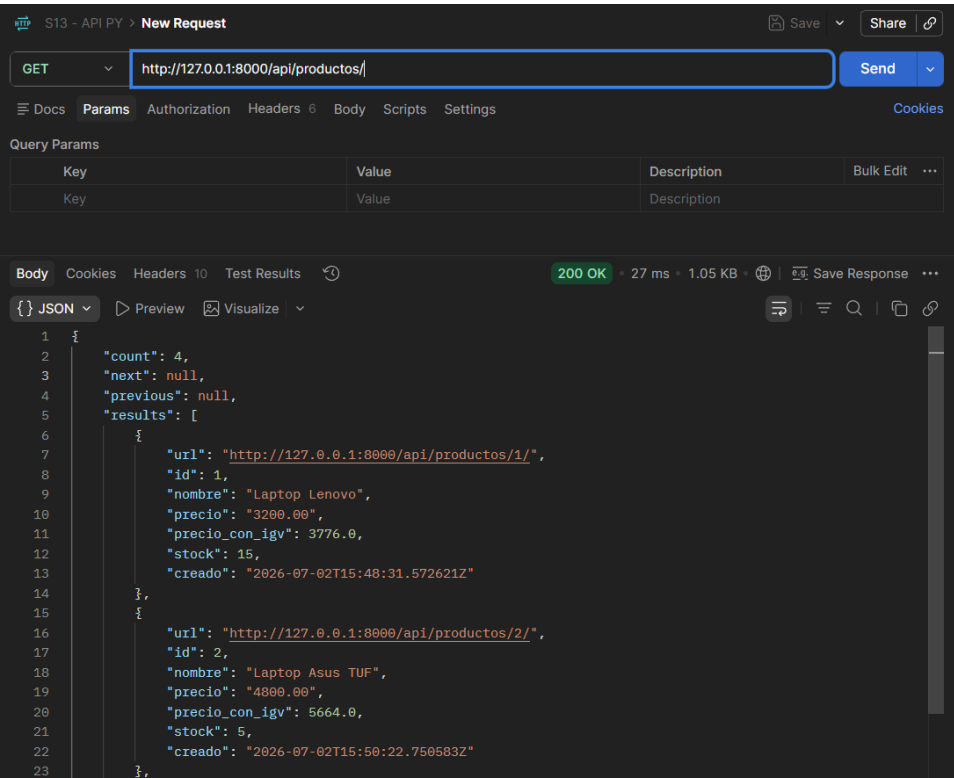
/api/productos/



Consultar

GET

/api/productos/



Buscar

GET

/api/productos/?search=Lenovo

API PYNew Request

SaveShare

GEThttp://127.0.0.1:8000/api/productos/?search=Lenovo

Send

DocsParamsAuthorizationHeaders6BodyScriptsSettings

Cookies

Query Params

Key	Value	Description	Bulk Edit	...
search	Lenovo			
Key	Value	Description		

Body

CookiesHeaders10Test Results

200 OK27 ms564 BSave Response

JSONPreviewVisualize

```
1 {
2   "count": 1,
3   "next": null,
4   "previous": null,
5   "results": [
6     {
7       "url": "http://127.0.0.1:8000/api/productos/1/",
8       "id": 1,
9       "nombre": "Laptop Lenovo",
10      "precio": "3200.00",
11      "precio_con_igv": 3776.0,
12      "stock": 15,
13      "creado": "2026-07-02T15:48:31.572621Z"
14    }
15  ]
16 }
```

Ordenar

GET

/api/productos/?ordering=-precio

GEThttp://127.0.0.1:8000/api/productos/?ordering=-precio

Send

DocsParamsAuthorizationHeaders6BodyScriptsSettings

Cookies

Query Params

Key	Value	Description	Bulk Edit	...
ordering	-precio			
Key	Value	Description		

Body

CookiesHeaders10Test Results

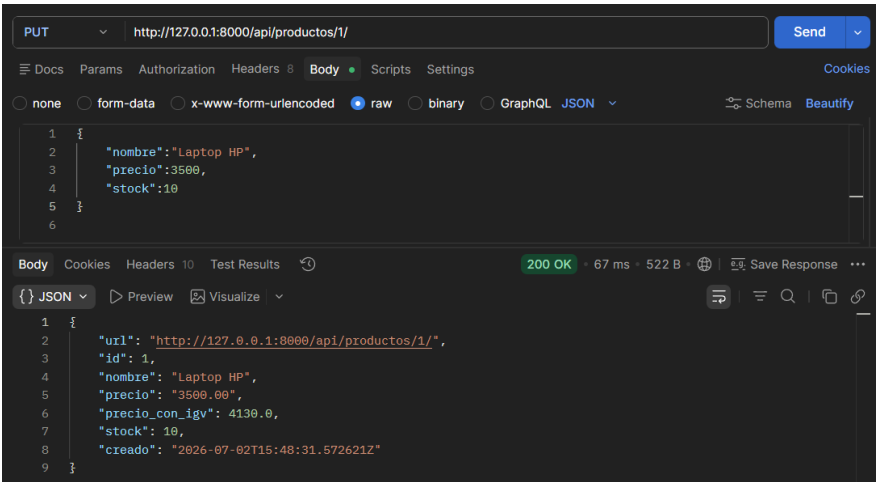
200 OK36 ms1.05 KBSave Response

JSONPreviewVisualize

```
1 {
2   "count": 4,
3   "next": null,
4   "previous": null,
5   "results": [
6     {
7       "url": "http://127.0.0.1:8000/api/productos/2/",
8       "id": 2,
9       "nombre": "Laptop Asus TUF",
10      "precio": "4800.00",
11      "precio_con_igv": 5664.0,
12      "stock": 5,
13      "creado": "2026-07-02T15:50:22.750583Z"
14    },
15     {
16       "url": "http://127.0.0.1:8000/api/productos/1/",
17       "id": 1,
18       "nombre": "Laptop Lenovo",
19       "precio": "3200.00",
20       "precio_con_igv": 3776.0,
21       "stock": 15,
22       "creado": "2026-07-02T15:48:31.572621Z"
23     },
24     {
25       "url": "http://127.0.0.1:8000/api/productos/3/",
26       "id": 3,
27       "nombre": "Samsung",
28       "precio": "3200.00",
29       "precio_con_igv": 3776.0,
```

Actualizar

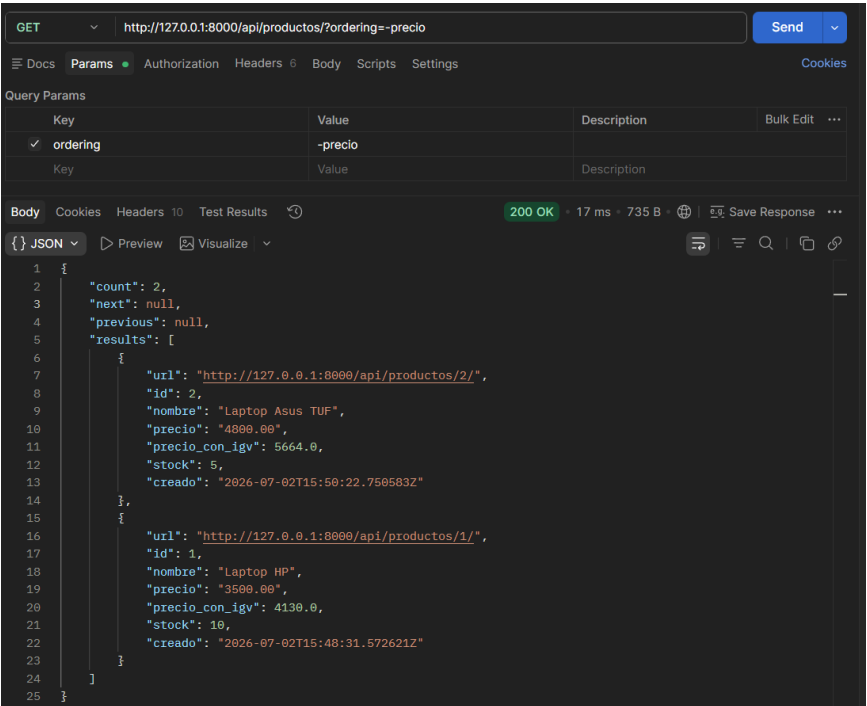
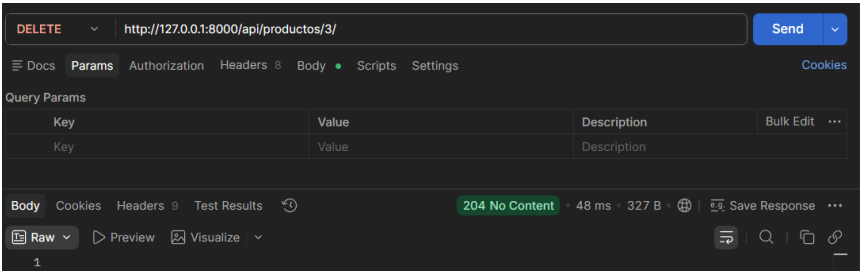
PUT



Eliminar

DELETE

/api/productos/4/



Conclusión

Se implementó una API RESTful siguiendo buenas prácticas de Django REST Framework. Se utilizaron serializadores avanzados, ModelViewSet y DefaultRouter para reducir código repetitivo, filtros, paginación y throttling para mejorar el rendimiento y la seguridad, configuración de CORS y CSRF para integrarse con aplicaciones SPA, y documentación automática mediante OpenAPI/Swagger, obteniendo una API lista para ser consumida por aplicaciones web o móviles.

Github

[Ichimaru465/S13-APIrestful](#)